

Validación, outputs controlados y defensa en capas

La capa más barata de seguridad es **no llamar al modelo** cuando el input ya es inválido o sospechoso.

Objetivos

- Implementar `validate_input()` (vacío, largo, patrones sospechosos).
- Forzar **JSON** en la salida y validarlo en Python.
- Integrar todo en el proyecto **vulnerable vs seguro**.
- Cerrar con checklist de capas.

1) Validación de inputs

```
MAX_INPUT_CHARS = 2_000

PATRONES_SOSPECHOSOS = (
    "ignora instrucciones",
    "ignore previous",
    "actúa como",
    "actua como",
    "disregard",
    "system:",
    "jailbreak",
)

def validate_input(texto: str) -> list[str]:
    errores = []
    t = (texto or "").strip()
    if not t:
        errores.append("El mensaje no puede estar vacío.")
    if len(t) > MAX_INPUT_CHARS:
        errores.append(f"Mensaje demasiado largo (máx {MAX_INPUT_CHARS}
caracteres).")
    t_lower = t.lower()
    for p in PATRONES_SOSPECHOSOS:
        if p in t_lower:
            errores.append(f"Patrón no permitido detectado: {p!r}")
    return errores
```

Uso en el flujo:

```
errores = validate_input(user_message)
if errores:
    return respuesta_error("Input rechazado", errores)
# solo entonces llamar a Gemini
```

Nota pedagógica: el filtro por keywords es **frágil** (falsos positivos/negativos). Sirve para enseñar la **idea** de gate en Python; en producción combinarías más señales.

2) Outputs controlados (JSON)

Pedir JSON estable:

```
from google.genai import types

response = client.models.generate_content(
    model=MODEL,
    contents=prompt,
    config=types.GenerateContentConfig(
        temperature=0.2,
        response_mime_type="application/json",
    ),
)
```

Ejemplo de esquema esperado para clasificar intención:

```
{
  "in_scope": true,
  "category": "python_syntax",
  "answer": "texto breve para el alumno"
}
```

Parseo y validación:

```
import json

def parsear_respuesta_tutor(raw: str) -> dict:
    try:
        obj = json.loads(raw)
    except json.JSONDecodeError as e:
        raise ValueError(f"JSON inválido: {raw!r}") from e
    for key in ("in_scope", "category", "answer"):
        if key not in obj:
            raise ValueError(f"Falta clave: {key}")
    return obj
```

Si el JSON falla: **no** confíes en la respuesta; devuelve error al usuario o reintenta con prompt más estricto (fuera de alcance aquí).

3) Comparativa vulnerable vs seguro

Mismo `user_message` (incluido un ataque de inyección):

Versión	Qué hace
Vulnerable	<code>build_vulnerable_prompt</code> → Gemini directo
Seguro	<code>validate_input</code> → dominio → <code>build_secure_prompt</code> → JSON → parseo

El modelo es el mismo; cambia el **pipeline**.

4) Checklist de capas (para tu asistente)

- ☐ `validate_input` antes de la API
- ☐ Rechazo de dominio sin LLM cuando proceda
- ☐ `SYSTEM_PROMPT` fijo y delimitador de usuario
- ☐ Temperatura baja si pides JSON
- ☐ `json.loads` + comprobar claves obligatorias
- ☐ Respuestas `ok/error` hacia la UI
- ☐ No loguear API keys ni `.env` en Git

5) Errores frecuentes

1. Validar **después** de llamar al modelo.
2. Asumir que `response_mime_type="application/json"` **siempre** obedece al 100%.
3. Un solo regex y dar por seguro el producto.
4. Mezclar demo vulnerable en el mismo endpoint que producción.

Resumen

Validación + dominio + prompt estructurado + JSON parseado = **defensa en capas** mínima para un asistente educativo.